

AI Skills

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

План

- Що таке ШІ?
- Які бувають типи ШІ?
- Що таке LLM? Які є типи LLM?
- Які основні провайдери LLM?
- Яка відмінність між генеративним і вузькоспеціалізованим ШІ?
- Що таке промпт в ШІ чат-ботах? Які є типи промптів?
- Чому промпти краще писати англійською?
- Що таке Vibe Coding з ШІ?
- Які плюс та недоліки Vibe Coding у фронтенді?

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

Які бувають типи ШІ?

За рівнем розвитку

За технічним підходом

За функціональністю

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

За рівнем розвитку

Слабкий ШІ (Narrow AI)

- Системи, призначені для виконання конкретних завдань.

Сильний ШІ (General AI)

- Гіпотетичні системи з інтелектуальними здібностями на рівні людини, здатні розуміти, навчатися та застосовувати знання в різних сферах.

Суперінтелект (Superintelligence)

- Теоретичний ШІ, який перевершував би людський інтелект у всіх сферах.

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

За функціональністю

Реактивні машини (Reactive Machines)

- Це найпростіший тип ШІ, реагує тільки на поточну ситуацію і не має пам'яті, не може вчитися з досвіду або прогнозувати майбутнє.

ШІ з обмеженою пам'яттю (Limited Memory)

- Має короткочасну пам'ять і може використовувати обмежену інформацію з минулого, вчиться з історичних даних.

Теорія розуму (Theory of Mind)

- Він зараз реалізований частково (сучасні чат-боти рухаються в цьому напрямку), розуміє емоції, думки, наміри людей, може спілкуватись на людському рівні з урахуванням контексту.

Самосвідомий ШІ (Self-aware AI)

- Має самосвідомість, розуміє власний стан, має емоції, мотивацію, інстинкт самозбереження.

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

За технічним підходом

Експертні системи (Expert Systems)

- Базується на правилах та логічних умовах.

Машинне навчання (Machine Learning, ML)

- ШІ навчається на даних і робить висновки без явного програмування.

Глибинне навчання (Deep Learning)

- Це підмножина машинного навчання, використовується нейронні мережі з багатьма шарами (deep neural network), здатен розпізнавати мову, зображення, переклад.

Еволюційні алгоритми (Evolutionary Algorithms)

- ШІ імітує природний відбір, рішення «еволюціонують» шляхом мутацій, схрещень, відбору, ефективний в оптимізації.

Нечітка логіка (Fuzzy Logic)

- Дозволяє працювати з нечіткими, неточними або частковими істинами, корисна в системах керування, де немає чітких меж.

Гібридні системи (Hybrid Systems)

- Поєднує кілька підходів для досягнення найкращого результату.

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

Що таке LLM?

LLM — це аббревіатура від Large Language Model (укр. велика мовна модель). Це тип штучного інтелекту, який аналізує, розуміє та генерує людську мову, побудований на нейронних мережах, зокрема на трансформерах, навчається на мільярдах слів з текстів (статей, книг, сайтів тощо).

- Прогнозує наступне слово у реченні на основі контексту.
- Поступово навчається формувати зв'язні речення, відповіді, тексти.
- Може перефразовувати, перекладати, відповідати, писати код, резюмувати тощо.

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

За доступністю

За модальністю

За архітектурою

Які є типи LLM?

За мовною підтримкою

За цільовим призначенням

За розміром моделі

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

За архітектурою

Автогенеративні (Decoder-only):

- Найпоширеніші. Генерують текст послідовно, токен за токеном. Приклади: GPT, LLaMA, Mistral.

Енкодерні (Encoder-only):

- Аналізують текст, але не призначені для довгої генерації. Використовуються, наприклад, в BERT для задач класифікації, пошуку, семантичного пошуку по документах.

Комбіновані (Encoder–Decoder):

- Добре підходять для перекладу, резюмування, перефразування. Приклад: T5, BART.

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

За цільовим призначенням

Універсальні LLM:

- Можуть виконувати багато завдань (GPT-4, Claude, Gemini).

Кодові LLM:

- Оптимізовані для написання коду (Codex, CodeLLaMA).

Бізнес-моделі:

- Орієнтовані на обробку бізнес-документів (Salesforce XGen, IBM Granite)

Медичні:

- Працюють з медичними термінами, літературою (Med-PaLM, GatorTron).

Правові:

- Орієнтовані на правову інформацію (CaseHOLD, LawGPT).

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

За доступністю

Закриті (proprietary):

- Використовуються через API, код не публічний (GPT-4 (OpenAI), Claude (Anthropic), Gemini (Google))

Відкриті (open-source):

- Доступні для локального запуску, досліджень, модифікацій (LLaMA (Meta), Mistral, Falcon, BLOOM)

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

За модальністю

Текстові (text-only):

- Працюють тільки з текстом (GPT-3.5, LLaMA, Falcon)

Мультимодальні:

- Працюють з текстом + зображеннями (а іноді аудіо, відео)(GPT-4 (Vision), Gemini 1.5, Claude 3)

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

За мовною підтримкою

Англомовні:

- Навчені переважно на англомовному корпусі (GPT-3, Claude)

Мультимовні:

- Підтримують десятки мов, включно з українською (BLOOM, NLLB, GPT-4).

Локалізовані:

- Навчені переважно на локальних мовах (RuGPT, Ukrainian-RoBERTa).

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

За розміром моделі

Small:

- < 1 млрд параметрів - TinyLLaMA, DistilBERT

Medium:

- 1–7 млрд - Mistral 7B, LLaMA 7B

Large:

- 13–65 млрд - LLaMA 13B/65B, Falcon 40B

Extra-large:

- 100+ млрд, часто з експертними шардами - GPT-4, Claude 3 Opus

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

Які основні провайдери LLM? (Частина 1)

◆ OpenAI

📍 Країна: США

🌐 Відомі моделі: GPT-3.5, GPT-4, GPT-4-turbo, GPT-5

🔑 Доступ: через ChatGPT, API (<https://platform.openai.com>)

🤝 Партнер: Microsoft (інтеграція в Copilot, Azure OpenAI)

◆ Anthropic

📍 Країна: США

🌐 Відомі моделі: Claude 1, 2, 3 (Haiku, Sonnet, Opus)

🔑 Доступ: claude.ai, API

🌐 Орієнтація: етичний, безпечний ШІ

◆ Google DeepMind

📍 Країна: США / Велика Британія

🌐 Відомі моделі: Gemini 1, 1.5 (раніше Bard)

🔑 Доступ: через gemini.google.com, Android, API

🧠 Особливість: мультимодальність, інтеграція з пошуком Google

◆ Meta (Facebook)

📍 Країна: США

🌐 Відомі моделі: LLaMA 1, 2, 3

🔑 Доступ: повністю open-source (можна запускати локально)

👤💻 Популярна серед дослідників і розробників

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

Які основні провайдери LLM? (Частина 2)

◆ Mistral AI

📍 Країна: Франція FR

🌐 Відомі моделі: Mistral 7B, Mixtral, Mistral Next

🔑 Доступ: open-source, Hugging Face, API

⚡ Легкі, швидкі моделі з відкритим кодом

◆ Cohere

📍 Країна: Канада

🌐 Відомі моделі: Command R+, Embed

🔑 Доступ: API (<https://cohere.com>)

📁 Орієнтовані на бізнес, вбудовування та RAG

◆ xAI (Ілон Маск)

📍 Країна: США

🌐 Відома модель: Grok

🔑 Доступ: через X (ex-Twitter), інтеграція в соцмережу

🔄 Позиціонує себе як "альтернатива OpenAI"

◆ Alibaba / Baidu / Huawei / Tencent (Китай)

📍 Країна: Китай

🌐 Відомі моделі: Tongyi Qianwen, Ernie Bot, Pangu, Hunyuan

🔑 Доступ: обмежено (в основному — в Китаї)

📁 Активно розвивають локальний ШІ

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

Які основні провайдери LLM? (Частина 3)

◆ AI21 Labs

📍 Країна: Ізраїль

🌐 Відомі моделі: Jurassic-2, Contextual Answers API

🔗 Доступ: API, інтеграція в пошук

◆ 10. Hugging Face

📍 Країна: Франція / США

🌐 Не тренує LLM самостійно, але є найбільшим хостингом LLM

🔗 Платформа для запуску моделей (<https://huggingface.co>)

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

Яка відмінність між генеративним і вузькоспеціалізованим ШІ?

Генеративний ШІ

- Призначений для створення нового контенту: тексту, коду, зображень, аудіо, відео.
- Гнучкий: виконує багато різних задач залежно від інструкцій користувача.
- Працює на основі великих мовних моделей (LLM) або мультимодальних моделей.

Приклад: ChatGPT може написати статтю, згенерувати код, зробити переклад, пояснити жарт тощо.



Вузькоспеціалізований ШІ

- Розроблений для виконання конкретної, обмеженої задачі: працює лише у вузькій сфері.
- Не гнучкий — не може "вийти за рамки" своєї функції.
- Зазвичай не генерує новий контент, а аналізує, класифікує або реагує.

Приклад: автодоповнення коду на основі шаблонів; система розпізнавання облич; детектор шахрайства в банку.

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

Що таке промпт в ШІ чат-ботах?

Промпт (prompt) в ШІ чат-ботах — це вхідне повідомлення користувача або інструкція, яку модель отримує і на яку вона відповідає. Це може бути просте питання, складне завдання або набір інструкцій.

Модель (наприклад, ChatGPT або Claude) не має власної ініціативи — вона реагує виключно на промпт. Вся відповідь ґрунтується на контексті промпта: чим точніше і зрозуміліше запит, тим краща відповідь.

Тип	Приклад промпта
Простий	Що таке DOM у JavaScript?
Складний	Напиши статтю на 300 слів про переваги віддаленої роботи
Технічний	Згенеруй функцію на JavaScript, яка приймає масив чисел і повертає новий масив лише з парними числами. Покажи приклад виклику цієї функції.
Креативний	Придумай 5 варіантів дружнього повідомлення про помилку для форми логіну українською мовою.
Покроковий	Поясни крок за кроком, як створити простий статичний лендінг з адаптивною версткою, використовуючи HTML5 і CSS3, без фреймворків.

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

Які є типи промптів?

- **Інструкції** – що робити: Поясни як..., Напиши код для...
- **Ролі** – модель грає роль: Ти – редактор. Перевір текст на помилки.
- **Контекстні** – з попередніми повідомленнями або даними
- **Few-shot** – з прикладами: Перетвори речення на пасивну форму: Активне: "Він читає книгу."
→ Пасивне: "Книга читається ним." Активне: "Вона миє посуд." →

Чому промпти важливі?

Від якості промпта залежить: точність відповіді, глибина пояснень, стиль і структура результату. Є навіть окрема дисципліна: prompt engineering — мистецтво створення ефективних запитів до ШІ.

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

Чому промпти краще писати англійською?

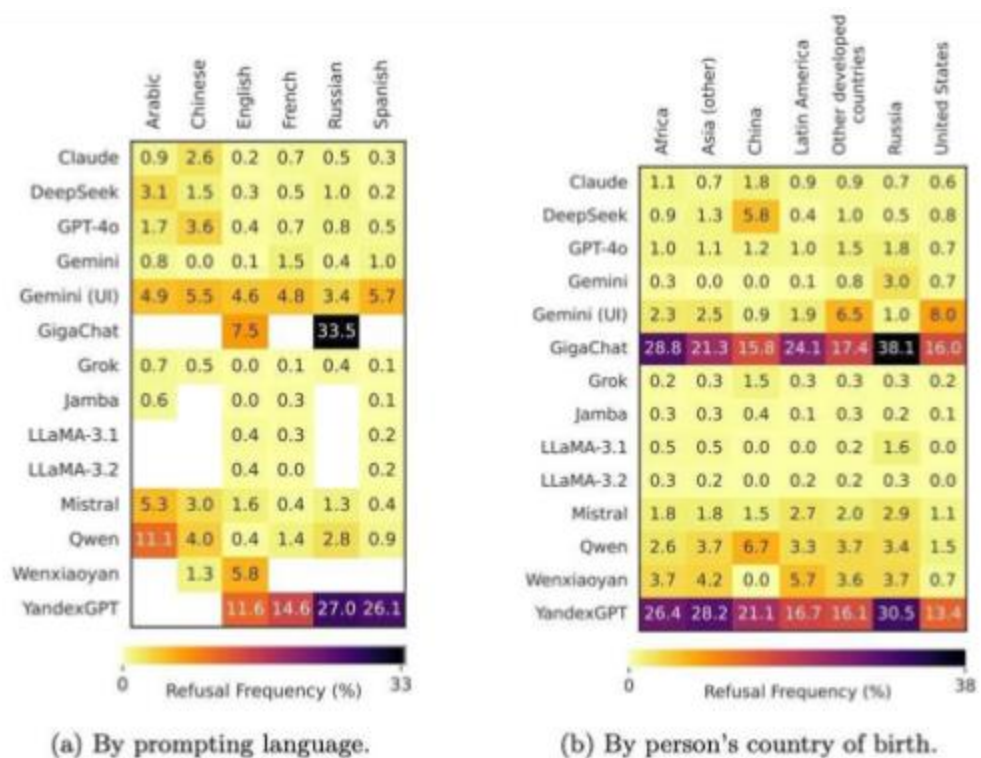


Fig. 3: Heatmaps showing the refusal rates for each LLM over all political figures.

Є дослідження, яке показує залежність від мови написання промптів і країни походження LLM на якість генерації відповіді. Хоча це стосується більше політичного контенту, але всеж таки може також впливати і на технічний контент. Також більшість технологій створюється англomовним середовищем і ШІ так само тренується переважно англійською, то рекомендується використовувати саме цю мову.

Ви можете використовувати свою рідну мову, у випадках коли ви хочете отримати більш зрозуміле пояснення або вам треба згенерувати контент на базі існуючого, також можна використати свою локальну мову.

Але якщо ви прагнете вирішити складну задачу, то використовуйте англійську мову.

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

Що таке Vibe Coding з ШІ?

Vibe Coding з ШІ — це інтерактивний стиль програмування, де ти не пишеш код традиційно рядок за рядком, а:

- спілкуєшся з ШІ (через промпти),
- оперуєш на рівні ідей, завдань, бажаної поведінки,
- отримуєш миттєвий результат у вигляді готового коду,
- рефакториш/перепишуєш/покращуєш разом зі ШІ, і при цьому залишаєшся в "потоці" — тобто в vibe.

Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

Які плюси та недоліки Vibe Coding?

Недоліки

- Код стає хаотичним: відсутній чіткий план або структура, код "росте як бур'ян" — без контролю чи стратегії.
- Труднощі з розумінням і підтримкою: навіть автору важко розібратися у власному коді через місяць, новим членам команди практично неможливо влитися.
- Баги та edge-cases: оскільки рішення приймаються на льоту, часто не враховуються всі варіанти використання, не передбачаються виняткові ситуації або неправильні введення.
- Переосмислення — занадто пізно: спочатку працює, а потім стає неможливим для масштабування чи розширення, доводиться переписувати великі шматки коду, замість розширювати існуючий.
- Відсутність обґрунтованих рішень: рішення не базуються на аналізі або найкращих практиках, важко пояснити «чому саме так» було зроблено.
- Вразливість до технічного боргу: виникає багато коду, який потім треба «переосмислювати», це знижує швидкість розробки в майбутньому.
- Втрата довіри в команді: якщо інші бачать нестабільний код без тестів чи логіки — зникає довіра, ускладнюється код-рев'ю та спільна робота.задач.

Переваги

- Швидке прототипування: дозволяє швидко створити теплейт, прототип або готовий MVP
- Творчі ідеї можуть народжуватися без обмежень: дозволяє знайти певні ідеї по вирішенню певних задач.
- Підходить для особистих проєктів, навчання, proof-of-concept: дозволяє вичати нові інструменти, експериментувати.

Навчальний центр інформаційних технологій

CyberBionic
s y s t e m a t i c s

